



PHASE 2 | TASK 1

Kafka Up and Events Flowing

Live AtliQ order events streaming from a Python producer into a Confluent Cloud topic



STEP 1

Create the Kafka Cluster

WHAT

Created a Confluent Cloud cluster, atliq-kafka, on the Basic tier in Azure Central India. Confluent manages the brokers, replication and configuration.

WHY

Confluent is managed Kafka, the way Databricks is managed Spark: the job is to create a cluster, a topic and route data, not to run servers. Basic has no fixed hourly base charge and uses a public endpoint, which fits a learning workload and lets both the laptop producer and Databricks connect.

STEP 1 | Create the Kafka Cluster

CONFLUENT

Search

🔔

?

☰

Home > Environments > default > Clusters > atliq-kafka >

Cluster

atliq-kafka

Cluster

Stream Lineage

Topics

Tableflow

Connectors

Clients

Kafka Streams

Flink

Schema Registry

CLI and Tools

atliq-kafka

Running Basic lkc-81qmdqq + Add owner Oct. 1 2026 12:09 PM Oct. 1 2026 12:09 PM + Add tags

+ Add description

Overview Monitor Networking API keys Settings Details

ID lkc-81qmdqq Environment default Provider AZURE Region centralindia

Production 0B/s Consumption 0B/s Storage 0B

Topics 0 Partitions 0

REST endpoint https://pkc-7prvp.centralindia. Bootstrap server pkc-7prvp.centralindia.azure.cc

Period Last hour Refresh

Select Anchor Resource Topic View full lineage

Configure a Client

Create Sample Data

Add connector

Add Topic

Get more data moving through your cluster

Integrate your cluster to the most popular data

Build Real-time Pipelines with Confluent Cloud

Join this hands-on workshop to learn how to stream

atliq-kafka: Running, Basic, Azure centralindia, with its bootstrap server.



STEP 2

Create the Topic

WHAT

Created the topic `atliq.orders.events` with 6 partitions and the default 7-day retention. Infinite retention and a Schema Registry schema were both left off.

WHY

Partitions give parallelism; each one is an ordered, append-only log. Seven days is the replay window, because the Bronze Delta table is the permanent record. The schema is applied on read in Silver, so Bronze can keep the raw JSON exactly as received.

STEP 2 | Create the Topic

CONFLUENT

Search

🔔

?

☰

Home > Environments > default > Clusters > atliq-kafka > Topics >

Cluster
atliq-kafka

Cluster

Stream Lineage

Topics

Tableflow

Connectors

Clients

Kafka Streams

atliq.orders.events

Actions

Query with Flink

Enable Tableflow

+ Add owner

Oct. 1 2026 12:14 PM

+ Add tags

+ Add description

Overview

Messages

Monitor

Schema

Clients

Settings

Details

Environment
default

Cluster
atliq-kafka

Provider
AZURE

Region
centralindia

Production
--

Consumption
0B/s

Partitions
6

Period Last hour

Refresh

View full lineage

Configure a Client

Add connector

topic
atliq.orders.events

Bring your topics to Data Lake

Expose this topic as an Apache Iceberg table to easily populate your Data Lakes. What is an Apache Iceberg table? [Find out here](#)

Enable Tableflow

No messages have been produced to this topic yet

Produce your first message, add a connector, or build a consumer application with a client.

Produce new message

Schema

Create a schema to add structure and compatibility to your data streams.

Create schema

Flink

Schema Registry

CLI and Tools

🗨

Topic atliq.orders.events: 6 partitions, Azure centralindia.



STEP 3

Complete the Producer

WHAT

Completed the three TODOs: a SASL_SSL producer configured from `.env` with `acks=all`, each event published keyed by `order_id` with a delivery callback, and a flush before exit. Also made order IDs unique per run.

WHY

Credentials stay in a git-ignored `.env`, never in code. Keying by `order_id` sends one order's events to one partition, where Kafka guarantees order. `flush()` delivers anything still buffered, so stopping the script never loses events.

STEP 3 | Complete the Producer (1 of 2)

Task 1: producer completed (config, keyed publish, flush)

main

1 file changed

Filter files...

phase2-streaming/producer

order_event_producer.py

Search within code

phase2-streaming/producer/order_event_producer.py

+29 -15

@@ -38,11 +38,26 @@

38 38

39 39

40 40 def make_producer() -> Producer:

41 - # TODO 1: return a confluent_kafka.Producer configured for Confluent Cloud.

42 - # You need: bootstrap.servers, security.protocol=SASL_SSL,

43 - # sasl.mechanisms=PLAIN, sasl.username (API key), sasl.password (API secret).

44 - # Read the values from environment variables (see .env.example).

45 - raise NotImplementedError("TODO 1: build the Producer config")

41 + """Confluent Cloud producer. Credentials come from .env, never from code."""

42 + conf = {

43 + "bootstrap.servers": os.environ["KAFKA_BOOTSTRAP"],

44 + "security.protocol": "SASL_SSL", # encrypted connection

45 + "sasl.mechanisms": "PLAIN", # API key / secret login

46 + "sasl.username": os.environ["KAFKA_API_KEY"],

47 + "sasl.password": os.environ["KAFKA_API_SECRET"],

48 + "client.id": "atliq-order-producer",

49 + "acks": "all", # confirmed only when all replicas have it

50 + }

51 + return Producer(conf)

52 +

53 +

54 + def delivery_report(err, msg):

55 + """Called once per event when Kafka confirms (or rejects) it."""

56 + if err is not None:

57 + print(f"FAILED key={msg.key().decode()} error={err}")

58 + else:

59 + print(f"OK key={msg.key().decode()} -> {msg.topic()} "

60 + f"[partition {msg.partition()}] @ offset {msg.offset()}")

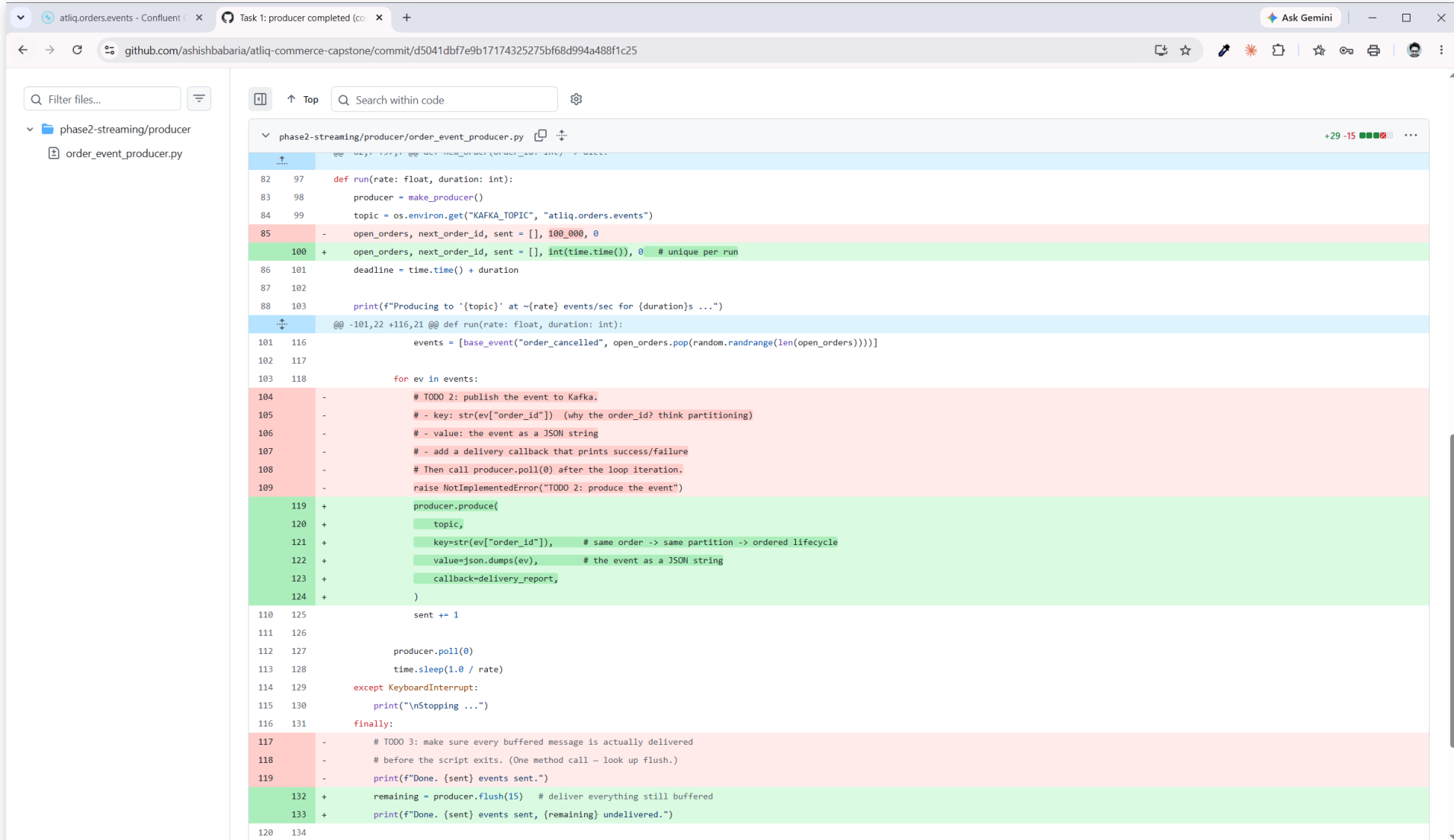
46 61

47 62

48 63 def now_iso() -> str:

Commit d5041db, TODO 1: the starter's NotImplementedError replaced by the SASL_SSL config and the delivery callback.

STEP 3 | Complete the Producer (2 of 2)



The screenshot shows a web browser displaying a GitHub repository for 'atliq-commerce-capstone'. The file 'order_event_producer.py' is open, showing a Python script for a Kafka producer. The script is divided into sections with colored backgrounds: blue for imports and initial setup, green for the main production logic, and red for TODO items and final cleanup. The script includes a 'run' function that takes 'rate' and 'duration' as arguments. It initializes a Kafka producer, sets the topic to 'atliq.orders.events', and generates a list of order IDs. The main loop produces events at a specified rate, using a key based on the order ID to ensure they are sent to the same partition. The script also includes a 'flush' call to ensure all buffered messages are delivered before exiting.

```
82 97 def run(rate: float, duration: int):
83 98     producer = make_producer()
84 99     topic = os.environ.get("KAFKA_TOPIC", "atliq.orders.events")
85 - open_orders, next_order_id, sent = [], 100_000, 0
100 + open_orders, next_order_id, sent = [], int(time.time()), 0 # unique per run
86 101     deadline = time.time() + duration
87 102
88 103     print(f"Producing to '{topic}' at ~{rate} events/sec for {duration}s ...")
@@ -101,22 +116,21 @@ def run(rate: float, duration: int):
101 116         events = [base_event("order_cancelled", open_orders.pop(random.randrange(len(open_orders))))]
102 117
103 118         for ev in events:
104 -             # TODO 2: publish the event to Kafka.
105 -             # - key: str(ev["order_id"]) (why the order_id? think partitioning)
106 -             # - value: the event as a JSON string
107 -             # - add a delivery callback that prints success/failure
108 -             # Then call producer.poll(0) after the loop iteration.
109 -             raise NotImplementedError("TODO 2: produce the event")
119 +             producer.produce(
120 +                 topic,
121 +                 key=str(ev["order_id"]), # same order -> same partition -> ordered lifecycle
122 +                 value=json.dumps(ev), # the event as a JSON string
123 +                 callback=delivery_report,
124 +             )
110 125         sent += 1
111 126
112 127         producer.poll(0)
113 128         time.sleep(1.0 / rate)
114 129     except KeyboardInterrupt:
115 130         print("\nStopping ...")
116 131     finally:
117 -         # TODO 3: make sure every buffered message is actually delivered
118 -         # before the script exits. (One method call - look up flush.)
119 -         print(f"Done. {sent} events sent.")
132 +         remaining = producer.flush(15) # deliver everything still buffered
133 +         print(f"Done. {sent} events sent, {remaining} undelivered.")
120 134
```

Commit d5041db, TODO 2 and 3: keyed produce() and flush() before exit, plus order IDs unique per run.



STEP 4

Stream Live Events

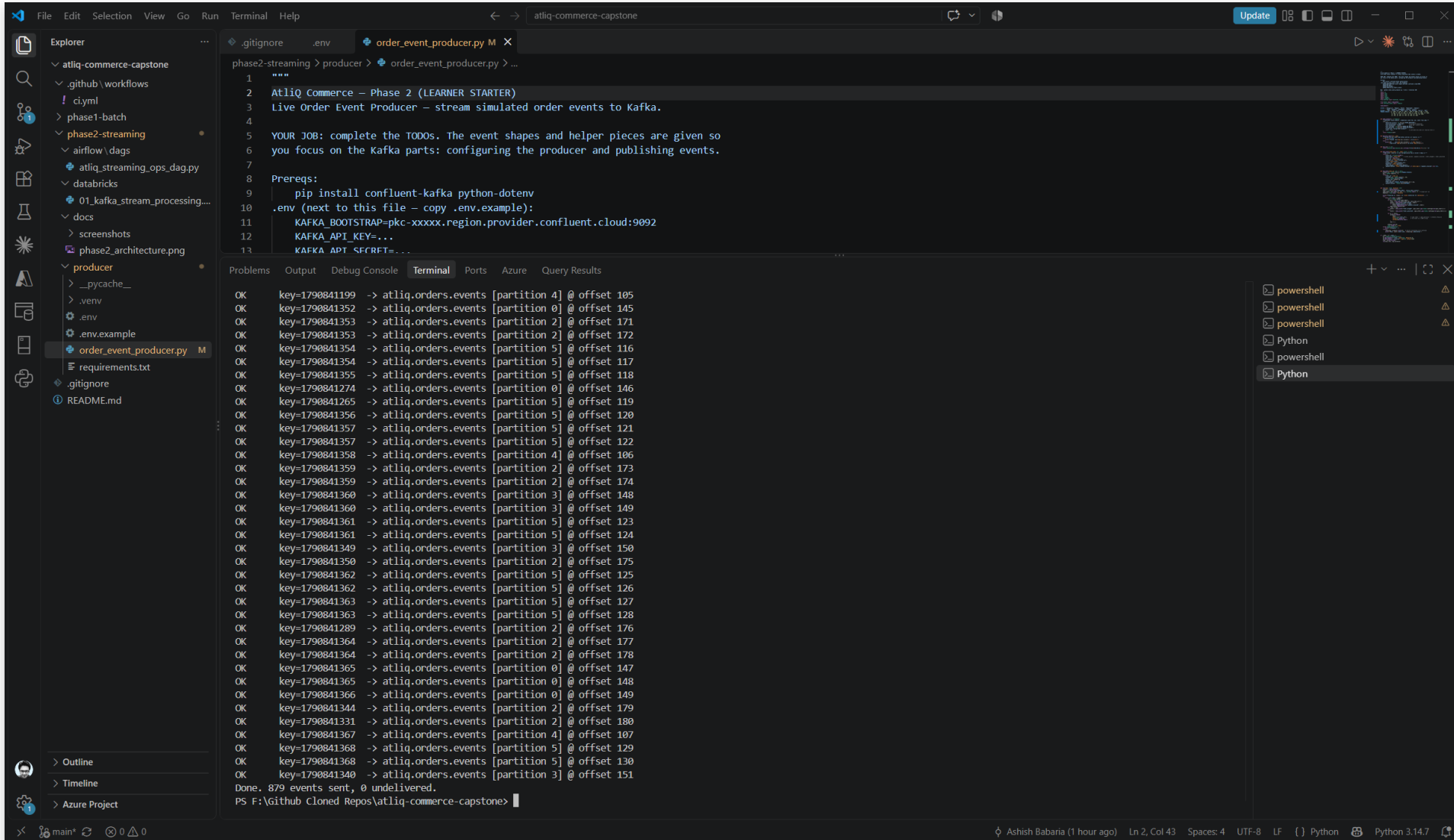
WHAT

Ran the producer for 5 minutes at about 2 events per second. Every event printed a delivery confirmation with its partition and offset, ending with 879 events sent and 0 undelivered.

WHY

The delivery callback is Kafka confirming each event, not the script assuming success. Repeated keys land on the same partition with rising offsets, for example key 1790841364 at partition 2, offsets 177 and 178.

STEP 4 | Stream Live Events



```
1 """
2 AtliQ Commerce – Phase 2 (LEARNER STARTER)
3 Live Order Event Producer – stream simulated order events to Kafka.
4
5 YOUR JOB: complete the TODOs. The event shapes and helper pieces are given so
6 you focus on the Kafka parts: configuring the producer and publishing events.
7
8 Prereqs:
9     pip install confluent-kafka python-dotenv
10     .env (next to this file – copy .env.example):
11     KAFKA_BOOTSTRAP=pkc-xxxxx.region.provider.confluent.cloud:9092
12     KAFKA_API_KEY=...
13     KAFKA_APT_SECRET=...
```

```
OK key=1790841199 -> atliq.orders.events [partition 4] @ offset 105
OK key=1790841352 -> atliq.orders.events [partition 0] @ offset 145
OK key=1790841353 -> atliq.orders.events [partition 2] @ offset 171
OK key=1790841353 -> atliq.orders.events [partition 2] @ offset 172
OK key=1790841354 -> atliq.orders.events [partition 5] @ offset 116
OK key=1790841354 -> atliq.orders.events [partition 5] @ offset 117
OK key=1790841355 -> atliq.orders.events [partition 5] @ offset 118
OK key=1790841274 -> atliq.orders.events [partition 0] @ offset 146
OK key=1790841265 -> atliq.orders.events [partition 5] @ offset 119
OK key=1790841356 -> atliq.orders.events [partition 5] @ offset 120
OK key=1790841357 -> atliq.orders.events [partition 5] @ offset 121
OK key=1790841357 -> atliq.orders.events [partition 5] @ offset 122
OK key=1790841358 -> atliq.orders.events [partition 4] @ offset 106
OK key=1790841359 -> atliq.orders.events [partition 2] @ offset 173
OK key=1790841359 -> atliq.orders.events [partition 2] @ offset 174
OK key=1790841360 -> atliq.orders.events [partition 3] @ offset 148
OK key=1790841360 -> atliq.orders.events [partition 3] @ offset 149
OK key=1790841361 -> atliq.orders.events [partition 5] @ offset 123
OK key=1790841361 -> atliq.orders.events [partition 5] @ offset 124
OK key=1790841349 -> atliq.orders.events [partition 3] @ offset 150
OK key=1790841350 -> atliq.orders.events [partition 2] @ offset 175
OK key=1790841362 -> atliq.orders.events [partition 5] @ offset 125
OK key=1790841362 -> atliq.orders.events [partition 5] @ offset 126
OK key=1790841363 -> atliq.orders.events [partition 5] @ offset 127
OK key=1790841363 -> atliq.orders.events [partition 5] @ offset 128
OK key=1790841289 -> atliq.orders.events [partition 2] @ offset 176
OK key=1790841364 -> atliq.orders.events [partition 2] @ offset 177
OK key=1790841364 -> atliq.orders.events [partition 2] @ offset 178
OK key=1790841365 -> atliq.orders.events [partition 0] @ offset 147
OK key=1790841365 -> atliq.orders.events [partition 0] @ offset 148
OK key=1790841366 -> atliq.orders.events [partition 0] @ offset 149
OK key=1790841344 -> atliq.orders.events [partition 2] @ offset 179
OK key=1790841331 -> atliq.orders.events [partition 2] @ offset 180
OK key=1790841367 -> atliq.orders.events [partition 4] @ offset 107
OK key=1790841368 -> atliq.orders.events [partition 5] @ offset 129
OK key=1790841368 -> atliq.orders.events [partition 5] @ offset 130
OK key=1790841340 -> atliq.orders.events [partition 3] @ offset 151
Done. 879 events sent, 0 undelivered.
PS F:\Github Cloned Repos\atliq-commerce-capstone>
```

Producer run: OK lines with partition and offset, then Done. 879 events sent, 0 undelivered.



STEP 5

Verify Events in Kafka

WHAT

Opened the topic's Messages tab while the producer ran and watched events arrive live, each with timestamp, partition, offset, key and JSON value.

WHY

This is independent proof from the Kafka side that events reached the topic. It also shows the keying at work: key 1790841179 has order_placed at partition 4, offset 51, then payment_received at partition 4, offset 52, one order in sequence.

STEP 5 | Verify Events in Kafka

CONFLUENT

Search

Home > Environments > default > Clusters > atliq-kafka > Topics >

Cluster

atliq-kafka

Cluster

Stream Lineage

Topics

Tableflow

Connectors

Clients

Kafka Streams

Flink

Schema Registry

CLI and Tools

atliq.orders.events

Actions

Query with Flink

Enable Tableflow

+ Add owner

Oct. 1 2026 12:14 PM

+ Add tags

+ Add description

Overview

Messages

Monitor

Schema

Clients

Settings

Details

Production in last hour

178 messages

Consumption in last hour

202 messages

Total messages

346

Retention time

1 week

Auto-refresh

Search...

More actions

Advanced filter

Preview

Search Mode

Quick

Full

Preview

Filter

By time

By offset

Latest

Partition

All

Max results

1,000

Timestamp	Partition	Offset	Key	Value
2026-10-01T07:52:38.587Z	4	51	1790841179	{ "event_id": "131722bc-e99c-4e15-a976-ef49a4cdd2fb", "event_type": "order_placed", "event_ts": "2026-10-01T07:52:38.587Z", ...
2026-10-01T07:52:38.587Z	4	52	1790841179	{ "event_id": "91b6ef25-a096-41d3-96c1-0644a62d5d9d", "event_type": "payment_received", "event_ts": "2026-10-01T07:52:38.587Z", ...
2026-10-01T07:52:38.087Z	3	74	1790841160	{ "event_id": "4123b36f-4051-4af8-88b6-dd1cf5a6749d", "event_type": "order_shipped", "event_ts": "2026-10-01T07:52:38.087Z", ...
2026-10-01T07:52:37.584Z	5	54	1790841135	{ "event_id": "eaea387e-aecf-4479-89fb-3d4938b4dccc", "event_type": "order_shipped", "event_ts": "2026-10-01T07:52:37.584Z", ...
2026-10-01T07:52:37.083Z	2	84	1790841088	{ "event_id": "fccd5513-960d-412e-b285-28ebc9570e1b", "event_type": "order_cancelled", "event_ts": "2026-10-01T07:52:37.083Z", ...
2026-10-01T07:52:36.582Z	2	82	1790841178	{ "event_id": "98c01827-3d44-4113-aa37-5271f8cc9d5e", "event_type": "order_placed", "event_ts": "2026-10-01T07:52:36.582Z", ...
2026-10-01T07:52:36.582Z	2	83	1790841178	{ "event_id": "b91eaf90-5152-4335-a3bd-6fcaebfe6e71", "event_type": "payment_received", "event_ts": "2026-10-01T07:52:36.582Z", ...
2026-10-01T07:52:36.081Z	5	52	1790841177	{ "event_id": "b018f614-7114-417f-b26b-28b46fabba26", "event_type": "order_placed", "event_ts": "2026-10-01T07:52:36.081Z", ...
2026-10-01T07:52:36.081Z	5	53	1790841177	{ "event_id": "6c03acbd-cc2e-48eb-a72c-59bb8f6a5878", "event_type": "payment_received", "event_ts": "2026-10-01T07:52:36.081Z", ...
2026-10-01T07:52:35.580Z	1	68	1790841176	{ "event_id": "678676c1-6de1-43fd-8c7b-57da95603d27", "event_type": "order_placed", "event_ts": "2026-10-01T07:52:35.580Z", ...
2026-10-01T07:52:35.580Z	1	69	1790841176	{ "event_id": "1375f355-98e5-4e40-8f01-1cc5c0ef889c", "event_type": "payment_received", "event_ts": "2026-10-01T07:52:35.580Z", ...

Page 1

<

>

Items per page

50

395 messages shown

Confluent Messages tab: live events with partition, offset, key (order_id) and JSON value.

Task 1 Complete

What this task delivered

Managed Kafka live

Confluent Cloud Basic cluster in Azure Central India with the topic atliq.orders.events: 6 partitions, 7-day replay window.

Secure, keyed producer

Credentials from a git-ignored .env, events keyed by order_id so each order's lifecycle stays in one partition, in order.

Delivery proven

Every event confirmed by Kafka through a delivery callback, and verified independently in the Confluent Messages tab.

Delivery proof

879

events sent

0

undelivered

6

partitions

7 days

replay window

Engineering decisions: Basic tier over Standard (no fixed hourly charge); no private networking (public endpoint for laptop and Databricks); acks=all for durability; schema on read in Silver instead of Schema Registry; .env git-ignored and verified with git check-ignore; order IDs seeded from time so runs never collide.